

系统工具开发课程实验报告（第一周）

廖世嘉 25020007073
25 级计算机科学与技术

2026 年 8 月 24 日

目录

1	实验基本信息	3
2	实验目的	3
3	实验环境与工具	3
4	实验一：含空格文件名的批量整理	3
4.1	实验要求	3
4.2	主要操作	3
4.3	结果与验证	4
5	实验二：随机访问日志统计	4
5.1	实验要求	4
5.2	脚本实现	4
5.3	结果	5
6	实验三：制造、解决并解释一次 Git 合并冲突	5
6.1	实验要求	5
6.2	实验过程	5
6.3	冲突原因与解决	6
7	实验四：修复并构建一页技术说明	6
7.1	实验要求	6
7.2	公式与交叉引用	6
7.3	表格与交叉引用	7
7.4	构建与验证	7
8	实验结果汇总	7
9	问题与解决方法	7
9.1	Windows 与 Linux 权限命令差异	7
9.2	LaTeX 中文编译问题	8
9.3	交叉引用检查方法	8

目录	2
10 实验总结	8

1 实验基本信息

表 1: 实验基本信息

项目	内容
姓名	廖世嘉
学号	25020007073
专业	25 级计算机科学与技术
实验环境	Linux / Bash / Git / XeLaTeX
实验日期	2026 年 8 月 24 日
GitHub 仓库	https://github.com/MrLLLiao/SystemToolkitDevCourse

2 实验目的

本实验围绕 Shell 基础与文件系统、Shell 管道与文本处理、Git 版本控制以及 LaTeX 文档编辑四个主题展开。通过四个连续实验，熟悉命令行环境下的文件操作、脚本编写、日志分析、分支管理、冲突解决和技术文档构建方法。同时，通过命令行验证文件权限、脚本退出码、Git 工作区状态以及 PDF 交叉引用，形成完整的”操作——验证——排错”实践流程。

3 实验环境与工具

本实验在 Linux 环境中完成，主要使用 Bash、find、ls、cp、chmod、awk、sort、head、Git、XeLaTeX/latexmk、pdftotext 与 file 等工具。实验目录为 SystemToolkitDevCourse，四个实验分别位于 q01、q02、q03 和 q04。

4 实验一：含空格文件名的批量整理

4.1 实验要求

在 q01 中创建 input/docs 和 input/tmp，并创建 notes one.txt（内容为 alpha 和 beta 两行）、隐藏文件.secret.txt（内容为 hidden）、空文件 empty.txt 以及两行日志。随后将所有.txt 文件复制到 work/25020007073，保持相对目录结构，并将目录权限设为 750、普通文件权限设为 640，最后生成 inventory.txt 列出相对路径和字节数。

4.2 主要操作

使用 mkdir -p 创建目录，利用 printf 和 touch 创建文件。对于包含空格的文件名，在 Shell 中使用引号包围整个路径：

```
printf "alpha\nbeta\n" > "input/docs/notes one.txt"
printf "hidden\n" > input/docs/.secret.txt
touch input/tmp/empty.txt
printf "INFO: program started\nINFO: program finished\n" > input/run.log
```

使用 `ls -laR input` 以长格式递归显示全部项目，其中 `-a` 保证隐藏文件能够显示。复制文件时使用 `find ... -print0` 配合 `read -d "`，避免文件名中的空格导致错误分割：

```
STUDENT_ID="25020007073"
mkdir -p "work/$STUDENT_ID"
find input -type f -name "*.txt" -print0 |
while IFS= read -r -d ' ' file; do
    relative="${file#input/}"
    mkdir -p "work/$STUDENT_ID/$(dirname "$relative")"
    cp -- "$file" "work/$STUDENT_ID/$relative"
done
```

然后使用 `find` 配合 `chmod` 设置权限：

```
find "work/$STUDENT_ID" -type d -exec chmod 750 {} +
find "work/$STUDENT_ID" -type f -exec chmod 640 {} +
```

生成清单：

```
find "work/$STUDENT_ID" -type f ! -name "inventory.txt" \
    -printf '%P\t%s\n' > "work/$STUDENT_ID/inventory.txt"
chmod 640 "work/$STUDENT_ID/inventory.txt"
```

4.3 结果与验证

实际检查结果表明，`input` 下正确存在 `notes one.txt`、`.secret.txt`、`empty.txt` 和 `run.log`。其中前三个文本文件的大小分别为 11、7 和 0 字节。复制后的 `work/25020007073` 中，目录显示为 `drwxr-x—`，即 750；普通文件显示为 `-rw-r—`，即 640。`inventory.txt` 成功记录复制后的相对路径和字节数：

```
tmp/empty.txt 0
docs/.secret.txt 7
docs/notes one.txt 11
```

5 实验二：随机访问日志统计

5.1 实验要求

在 `q02` 中创建给定的 CSV 数据并编写 `analyze.sh`。脚本接收 CSV 路径，若文件不存在则将错误写入标准错误并返回非零退出码；正常情况下使用 Shell 管道、`awk`、`sort`、`head` 统计 HTTP 5xx 次数最多的前两个 `path`，并计算全部数据行的平均延迟。

5.2 脚本实现

文件检查部分如下：

```
file="$1"
if [ -z "$file" ]; then
    echo "Error: missing CSV file path" >&2
    exit 1
fi
```

```
if [ ! -f "$file" ]; then
    echo "Error: file not found: $file" >&2
    exit 1
fi
```

5xx 统计通过 awk 对状态码进行过滤，再通过 sort 和 head 完成排序与截取：

```
awk -F',' 'NR > 1 && $4 ~ /^5[0-9][0-9]$/ { count[$3]++ }
END { for (path in count) print count[path], path }' "$file" |
sort -k1,1nr -k2,2 |
head -n 2
```

sort -k1,1nr 负责按次数数值降序排列，-k2,2 在次数相同时按路径字典序排列。平均延迟通过另一个 awk 忽略表头并保留两位小数：

```
awk -F',' 'NR > 1 { sum += $5; count++ }
END { printf "%.2f\n", sum / count }' "$file"
```

5.3 结果

运行./analyze.sh access.csv 得到：

```
Top 2 5xx paths:
2 /api/orders
2 /api/users
Average latency_ms:
193.75
```

运行不存在的文件时，脚本输出文件不存在错误并返回退出码 1，说明标准错误和非零退出状态处理符合要求。本实验未使用 Python、电子表格或手工统计。

6 实验三：制造、解决并解释一次 Git 合并冲突

6.1 实验要求

在 q03 中从零初始化仓库。在 main 中创建包含 mode=normal 的 config.txt 并提交；然后创建 feature-a 改为 mode=safe 并提交；从初始 main 创建 feature-b 改为 mode=fast 并提交。回到 main 后先合并 feature-a，再合并 feature-b，主动制造冲突并解决。

6.2 实验过程

初始化仓库：

```
git init
git branch -M main
printf "mode=normal\n" > config.txt
git add config.txt
git commit -m "Initial commit"
```

创建并提交 feature-a：

```
git switch -c feature-a
printf "mode=safe\n" > config.txt
git add config.txt
git commit -m "Set mode to safe"
```

回到初始 main 后创建并提交 feature-b:

```
git switch main
git switch -c feature-b
printf "mode=fast\n" > config.txt
git add config.txt
git commit -m "Set mode to fast"
```

回到主分支并按要求合并:

```
git switch main
git merge feature-a
git merge feature-b
```

6.3 冲突原因与解决

feature-a 将初始状态 mode=normal 改为 mode=safe, 而 feature-b 将同一行改为 mode=fast。当两个修改合并时, Git 无法自动判断应保留哪一个版本, 因此要求人工处理冲突。本实验按要求保留 mode=safe, 并增加 note=reviewed:

```
mode=safe
note=reviewed
```

随后执行:

```
git add config.txt
git commit -m "Merge feature-b and resolve conflict"
```

最终使用 git status 检查, 工作区显示 nothing to commit, working tree clean; 使用 git log -all -graph -decorate -oneline 能够观察到初始提交、两个特性分支提交以及最终合并提交, 说明版本历史完整。

7 实验四：修复并构建一页技术说明

7.1 实验要求

在 q04/report.tex 中加入带编号和标签的公式, 在正文中使用 \ref 或 \eqref 引用; 加入 2 行 2 列表格并设置 caption 和 label; 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 PDF, 并确认交叉引用没有显示“??”。

7.2 公式与交叉引用

实验采用如下公式表示总延迟:

$$L_{\text{total}} = \sum_{i=1}^n L_i \quad (1)$$

正文通过式 (1) 进行引用。由于公式包含 `equation` 环境和 `label`，LaTeX 能够为公式自动编号并建立交叉引用。

7.3 表格与交叉引用

实验中使用严格的 2 行 2 列表格：

表 2: 日志统计结果示例

Item	Value
Average latency	193.75 ms

正文通过表 2 引用该表，表格具有独立的编号、标题和标签。

7.4 构建与验证

原实验中使用 `pdflatex` 编译含中文作者信息的文档时出现 Unicode 字符错误。原因是 `pdfLaTeX` 对中文 Unicode 字符的直接支持有限。修正后采用 `CTeX/XeLaTeX` 方案生成文档。使用命令行构建：

```
latexmk -pdf -halt-on-error report.tex
```

生成 PDF 后使用 `pdftotext` 和 `grep` 检查交叉引用，结果显示 PDF 文件能够正常生成，正文中的公式引用显示为对应编号，表格引用显示为“Table 1”，而 `grep -Fn '??'` 无输出，说明交叉引用没有出现未解析的“??”。

8 实验结果汇总

表 3: 四个实验的完成情况

题号	实验主题	验证结果
1	Shell 文件系统	空格文件名、隐藏文件、复制、权限和 <code>inventory</code> 均正确
2	Shell 日志分析	前两名 <code>5xx path</code> 及平均延迟统计正确，异常文件返回非零状态
3	Git 版本控制	成功产生、解决冲突，最终工作区干净并可查看图形历史
4	LaTeX 文档构建	PDF 成功生成，公式和表格交叉引用正常，无“??”

9 问题与解决方法

9.1 Windows 与 Linux 权限命令差异

实验初期曾在 Windows PowerShell 中执行 `chmod`，系统提示命令不存在。由于题目要求使用 750 和 640 这类 Unix 权限模式，因此将相关操作放到 Linux 环境下，通过 `find` 和 `chmod` 完成。

9.2 LaTeX 中文编译问题

使用 pdfLaTeX 直接编译包含中文姓名的文档时出现 Unicode 字符错误。之后改用 CTeX/XeLaTeX, 使中文文本和标题能够正常排版。这一问题体现了不同 TeX 引擎在字体和 Unicode 支持方面的差异。

9.3 交叉引用检查方法

检查 PDF 文本中的”??”时, 应采用固定字符串搜索而不是容易产生歧义的正则表达式, 因此最终使用 `grep -Fn '??'` 进行验证。

10 实验总结

通过本次实验, 我系统练习了 Linux 命令行环境中的文件管理、权限控制、Shell 管道、文本分析、Git 分支管理和 LaTeX 文档构建。实验一使我熟悉了隐藏文件、空格文件名和权限模式; 实验二让我理解了多个 Unix 文本工具通过标准输入输出构成流水线的方式; 实验三让我实际经历了 Git 合并冲突从产生到解决的全过程; 实验四则强化了技术文档的结构化编辑、交叉引用和自动化构建能力。

本次实验中最重要的是: 命令行任务不仅要”做出来”, 还应当通过命令验证最终状态。例如, 文件权限需要通过 `ls -l` 或 `find` 检查, Shell 脚本需要检查退出码, Git 需要通过 `git status` 和日志确认状态, LaTeX 则需要检查生成的 PDF 与交叉引用。通过这些验证步骤, 可以显著降低操作错误并提高实验结果的可重复性。